

Université Internationale de Rabat

THE INNOVATIVE UNIVERSITY

Hands-on CUDA

2026-2027

I Ressources

— NVIDIA website (documentation, download, CUDA API Reference Manual, Programming Guide, CUDA C Best Practices Guide, ...) :
`http://developer.nvidia.com/nvidia-gpu-computing-documentation`
— CUDA SDK Code Samples :
`http://www.nvidia.com/object/cuda_get_samples.html`
— Textbooks on CUDA :
`J. Sanders, E. Kandrot, CUDA by Example: An Introduction to General-Purpose GPU Programming.`
`D. Kirk, W. Hwu, Programming Massively Parallel Processors: A Hands-on Approach.`
`W. Hwu (editor), GPU Computing Gems, Emerald Edition.`

II Lab1

Ex0 : CUDA Hello World

First, check https ://docs.nvidia.com/cuda/cuda-installation-guide-linux/ for installing the CUDA Toolkit. We will start by the basic "Hello world" example program (lab0), *cuda_ex_1.c*.

```
#include <stdlib.h>
#include <stdio.h>
int main()
{
printf("Hello World!\n");
return EXIT_SUCCESS;
}
```

To compile and run it, `nvcc cuda_ex_0.c -o cuda_ex_0`. This basic program is just standard C that runs on the host. NVIDIA ?s compiler (*nvcc*) will not complain about CUDA programs with no device code. To add device code, CUDA C keyword __global__ indicates that a function runs on the device and is called from host code, see *cuda_ex_0.cu*. *nvcc* splits source file into host and device components; NVIDIA's compiler handles device functions like kernel() and the standard host compiler handles host functions like *main()*.

```
#include <stdlib.h>
#include <stdio.h>
__global__ void kernel( void ) { }
int main()
```

```
{
kernel<<< 1, 1 >>>();
printf("Hello World!\n");
return EXIT_SUCCESS;
}
```

Triple angle brackets mark a call from host code to device code, Sometimes called a "kernel launch", which is required to execute a function on the GPU. The function *kernel()* does nothing in this example!

Ex1 : vector addition

Review and run the code of vector addition *vecadd.c* (lab1). The file *vecadd.cu* is its Cuda implementation, by adding required statements in order :

- to allocate/de-allocate device memory
- to exchange data between host/device memories
- to launch a GPU kernel

Add timing measurement code. When launching a CUDA kernel, the choice of grid and block configuration (e.g., $<<< N, 1 >>>$, $<<< 1, N >>>$, $<<< N/T, T >>>$) directly affects performance, scheduling, and resource utilization. For instance, ¡¡¡N, 1¿¿¿ launches N blocks, each containing 1 thread, and each block handles a single element. However, this requires very high number of blocks with huge scheduling overhead, since thread-level parallelism is not exploited inside blocks. Modern GPUs prefer, however, fewer blocks with more threads per block. Compare these three configurations.

III Lab2

The objective is to port the sum-reduction and matrix-matrix multiplication algorithms to the GPU in order to exploit massive parallelism and improve computational performance, as follows :

- Implement both algorithms on the CPU and GPU (using CUDA).
- Measure and compare the execution time for different data sizes.
- Compute the speedup achieved by the GPU implementation relative to the CPU.
- Analyze how data size and memory access patterns affect overall performance.
- Compare the three configurations ($<<< N, 1 >>>$, $<<< 1, N >>>$, $<<< N/T, T >>>$).

Ex2 : sum reduction

Review the C implementation of sum reduction, `sum.c`. The task is to port it to GPU. Compare the three configurations ($<<< N, 1 >>>$, $<<< 1, N >>>$, $<<< N/T, T >>>$).

Ex3 : matrix-matrix multiplication

Review the CPU implementation of `mmult.c`, matrix-matrix multiplication and `mmult.cu`, a template for GPU implementation. The task is to Port matrix-matrix multiplier to GPU. Compare the three configurations ($<<< N, 1 >>>$, $<<< 1, N >>>$, $<<< N/T, T >>>$).

IV Deliverables

Documents to be submitted (via the platform, see submission deadlines) : a report (maximum 5 pages), the source files (.cu files), and the test datasets. All materials should be submitted together in a single archive file(Lastname-firstname-TPx.zip/gz).